ParcelPilot Platform Architecture and Service Map

Service responsibilities, dependencies, request paths, configuration boundaries, and observability.

ParcelPilot operations reference

Contents

1. System overview

ParcelPilot is a fictional parcel-tracking application. Customers sign in, search for an order, view shipment events, download shipment documents, and manage notification preferences. Operations users also run reports over order and tracking activity.

The system is split into small services so one user-facing problem can be traced to the service that owns that part of the flow. The API service is the front door, but it does not own order records, carrier snapshots, files, notification queues, or analytics data.

Operational notes

- A healthy API process does not prove every downstream service is healthy.
- Troubleshooting should follow the failing route rather than inspect every service at once.
- One page in the UI may call several backend routes.

Example

When investigating system overview, compare the reported user action with the exact service or dependency that owns it. Record the evidence that supports the conclusion and avoid using unrelated warnings simply because they occur nearby in the logs.

What to record

Record the incident time, affected route, request identifier when available, service state, relevant error code, and the next check. This gives another operator enough context to continue without replaying the whole investigation.

Operational detail

ParcelPilot intentionally separates user journeys across services. This means a broad page-level symptom should be decomposed into the requests that make up the page before a service is blamed.

Dependencies should be followed in the direction the request actually travels. Start at the route, move to the owning service, then inspect the database, snapshot, file store, queue, or provider used by that service.

Example investigation

- Identify the failing request.
- Find the owning service.
- Check its current health and recent logs.
- Inspect only the configuration or data source that belongs to that path.
- Verify with a second request after any change.

This keeps troubleshooting understandable and reduces the chance that unrelated warnings turn into unnecessary changes.

2. API service

The API service validates requests, checks sessions, calls downstream services, and formats responses. It has service URLs for auth, orders, tracking, documents, notifications, and reporting.

A route can fail while GET /health still returns healthy because the basic health endpoint checks the API process, not a complete end-to-end order lookup.

Operational notes

- API-500 is generic and should be correlated with downstream logs.
- API-503 indicates a required dependency could not be used.
- A missing service URL is a configuration problem, not the same as a reachable service returning an error.

Example

When investigating api service, compare the reported user action with the exact service or dependency that owns it. Record the evidence that supports the conclusion and avoid using unrelated warnings simply because they occur nearby in the logs.

What to record

Record the incident time, affected route, request identifier when available, service state, relevant error code, and the next check. This gives another operator enough context to continue without replaying the whole investigation.

Normal operating pattern

A normal API request has a request_id, a route, a downstream call when needed, and a final status. Basic health normally completes in tens of milliseconds, while an order route includes the time spent waiting for the orders service.

Failure pattern

A generic API-500 is most useful when correlated with a downstream code on the same request_id. If the downstream service returns a specific error, keep that code visible in the incident record instead of treating API-500 as the final diagnosis.

Worked example

For req-0048123, API-500 at 14:06:18 followed an ORD-500 from the orders service under the same request identifier. That pattern points to the orders path rather than the API process itself.

Useful evidence to keep

- The exact incident time or best available time window.
- The route, service, and request_id when available.
- The specific error code rather than only the HTTP status.
- One known-good comparison case when practical.

The operator should also note what was checked and found to be normal. This prevents the next person from repeating the same work and helps keep an unrelated warning from becoming the assumed cause.

3. Authentication service

The authentication service validates credentials, returns session tokens, and applies temporary account locks after repeated failures. Login incidents should first be scoped to one user, a group, or all known-good accounts.

AUTH-401 is expected for invalid credentials. AUTH-423 means an account lock. AUTH-503 indicates service-level unavailability.

Operational notes

- Do not print passwords or session tokens.
- One locked user does not imply an auth outage.
- Multiple known-good users failing with AUTH-503 is stronger service evidence.

Example

When investigating authentication service, compare the reported user action with the exact service or dependency that owns it. Record the evidence that supports the conclusion and avoid using unrelated warnings simply because they occur nearby in the logs.

What to record

Record the incident time, affected route, request identifier when available, service state, relevant error code, and the next check. This gives another operator enough context to continue without replaying the whole investigation.

Normal operating pattern

Normal authentication traffic includes successful logins, occasional AUTH-401 responses for bad credentials, token refreshes, and a small number of expected account-lock events during testing.

Failure pattern

A service incident is more likely when known-good accounts fail together, AUTH-503 repeats across many request IDs, and the auth health state is degraded or down. A single AUTH-423 is an account-state problem until evidence shows broader impact.

Worked example

If one user is locked at 09:14 but two other known-good users sign in at 09:15, do not classify the event as an authentication outage.

Useful evidence to keep

- The exact incident time or best available time window.
- The route, service, and request_id when available.
- The specific error code rather than only the HTTP status.
- One known-good comparison case when practical.

The operator should also note what was checked and found to be normal. This prevents the next person from repeating the same work and helps keep an unrelated warning from becoming the assumed cause.

4. Orders service

The orders service owns order detail lookup and reads a local SQLite database. It requires ORDERS_DB_PATH, read access to the file, and a compatible schema version.

ORD-404 means an order is not present. ORD-500 means the database could not be opened. ORD-409 means the application and database schema versions disagree.

Operational notes

- Order details can fail while authentication still works.
- Tracking is a separate dependency from the basic order record.
- All order IDs failing is different from one missing order.

Example

When investigating orders service, compare the reported user action with the exact service or dependency that owns it. Record the evidence that supports the conclusion and avoid using unrelated warnings simply because they occur nearby in the logs.

What to record

Record the incident time, affected route, request identifier when available, service state, relevant error code, and the next check. This gives another operator enough context to continue without replaying the whole investigation.

Normal operating pattern

Normal order lookup reads the configured SQLite file, finds the requested order, and returns order details independently of tracking. ORD-404 for an unknown identifier is expected application behavior.

Failure pattern

ORD-500 DB_OPEN_FAILED points to the database path or file access. ORD-409 points to schema version mismatch. These failures can both become HTTP 500 responses at the API layer, but the corrective checks are different.

Worked example

If ORDERS_DB_PATH is missing and every tested order ID returns ORD-500, the missing database path is stronger evidence than a simultaneous tracking warning on another request.

Useful evidence to keep

- The exact incident time or best available time window.
- The route, service, and request_id when available.
- The specific error code rather than only the HTTP status.
- One known-good comparison case when practical.

The operator should also note what was checked and found to be normal. This prevents the next person from repeating the same work and helps keep an unrelated warning from becoming the assumed cause.

5. Tracking service

The tracking service reads simulated carrier data and returns shipment events. It can return partial data, stale data, invalid-snapshot errors, or provider timeouts.

TRK-206 means partial events, TRK-409 means stale data, TRK-422 means invalid snapshot structure, and TRK-504 means timeout.

Operational notes

- Check last_updated against incident time.
- Partial data should remain labeled partial.
- Repeated timeouts matter more than one slow lookup.

Example

When investigating tracking service, compare the reported user action with the exact service or dependency that owns it. Record the evidence that supports the conclusion and avoid using unrelated warnings simply because they occur nearby in the logs.

What to record

Record the incident time, affected route, request identifier when available, service state, relevant error code, and the next check. This gives another operator enough context to continue without replaying the whole investigation.

Normal operating pattern

Normal tracking reads a recent carrier snapshot and returns a sequence of shipment events. Some orders legitimately have fewer events than others, so the number of events alone is not an error.

Failure pattern

TRK-206 means the response is incomplete, TRK-409 means the snapshot is too old, TRK-422 means the source cannot be used safely, and TRK-504 means the carrier lookup exceeded its timeout.

Worked example

A snapshot updated eight minutes ago with two events for one order may be valid partial data. A snapshot updated two hours ago should be treated as stale even if the JSON structure is valid.

Useful evidence to keep

- The exact incident time or best available time window.
- The route, service, and request_id when available.
- The specific error code rather than only the HTTP status.
- One known-good comparison case when practical.

The operator should also note what was checked and found to be normal. This prevents the next person from repeating the same work and helps keep an unrelated warning from becoming the assumed cause.

6. Documents service

The documents service provides shipment labels, customs forms, proof-of-delivery files, and related documents when they exist.

DOC-404 is a missing file, DOC-403 is a file that exists but cannot be read, and DOC-500 means the configured document-store path is invalid.

Operational notes

- A missing optional document should not make order lookup fail.
- File permissions and store-path configuration are different problems.
- Keep troubleshooting scoped to the document route if order details still work.

Example

When investigating documents service, compare the reported user action with the exact service or dependency that owns it. Record the evidence that supports the conclusion and avoid using unrelated warnings simply because they occur nearby in the logs.

What to record

Record the incident time, affected route, request identifier when available, service state, relevant error code, and the next check. This gives another operator enough context to continue without replaying the whole investigation.

Normal operating pattern

Document routes first confirm the order and then look for the requested file under the configured store. Some orders do not have proof-of-delivery or customs documents, so a controlled not-found response can be normal.

Failure pattern

DOC-404, DOC-403, and DOC-500 should remain separate. A missing file, an unreadable file, and an invalid document root have different causes and different next checks.

Worked example

If label.pdf opens but proof.pdf returns DOC-404, do not change the global document-store path. Verify whether proof.pdf exists for that order.

Useful evidence to keep

- The exact incident time or best available time window.
- The route, service, and request_id when available.
- The specific error code rather than only the HTTP status.
- One known-good comparison case when practical.

The operator should also note what was checked and found to be normal. This prevents the next person from repeating the same work and helps keep an unrelated warning from becoming the assumed cause.

7. Notifications service

Notifications are processed asynchronously through a queue and a simulated delivery provider. A successful enqueue does not guarantee immediate delivery.

NTF-422 means a bad payload, NTF-429 indicates backlog above the operational threshold, and NTF-503 indicates provider unavailability.

Operational notes

- Queue depth and oldest pending age are useful evidence.
- A backlog can cause delay without affecting orders or tracking.
- Provider failure and queue backlog require different next checks.

Example

When investigating notifications service, compare the reported user action with the exact service or dependency that owns it. Record the evidence that supports the conclusion and avoid using unrelated warnings simply because they occur nearby in the logs.

What to record

Record the incident time, affected route, request identifier when available, service state, relevant error code, and the next check. This gives another operator enough context to continue without replaying the whole investigation.

Operational detail

ParcelPilot intentionally separates user journeys across services. This means a broad page-level symptom should be decomposed into the requests that make up the page before a service is blamed.

Dependencies should be followed in the direction the request actually travels. Start at the route, move to the owning service, then inspect the database, snapshot, file store, queue, or provider used by that service.

Example investigation

- Identify the failing request.
- Find the owning service.
- Check its current health and recent logs.
- Inspect only the configuration or data source that belongs to that path.
- Verify with a second request after any change.

This keeps troubleshooting understandable and reduces the chance that unrelated warnings turn into unnecessary changes.

8. Reporting service

The reporting service reads a separate analytics database. It supports operational volume and status reports and does not share the orders database.

RPT-422 is used for invalid or excessive date ranges, RPT-500 for query failure, and RPT-504 when execution exceeds the configured timeout.

Operational notes

- Large reports can be slower than small reports without an outage.
- Reporting problems should be isolated from transactional order lookup.
- Check date range before changing timeouts.

Example

When investigating reporting service, compare the reported user action with the exact service or dependency that owns it. Record the evidence that supports the conclusion and avoid using unrelated warnings simply because they occur nearby in the logs.

What to record

Record the incident time, affected route, request identifier when available, service state, relevant error code, and the next check. This gives another operator enough context to continue without replaying the whole investigation.

Normal operating pattern

Reporting reads a separate analytics database. Small date ranges normally complete faster than large ones, and the same report can return different row counts as the requested window changes.

Failure pattern

RPT-422 is a validation problem, RPT-500 is a query failure, and RPT-504 is a timeout after execution begins. A timeout should be investigated with date range, query type, and repeated duration.

Worked example

A 90-day report timing out while 7-day reports complete in 300 ms is not the same as every report failing immediately with RPT-500.

Useful evidence to keep

- The exact incident time or best available time window.
- The route, service, and request_id when available.
- The specific error code rather than only the HTTP status.
- One known-good comparison case when practical.

The operator should also note what was checked and found to be normal. This prevents the next person from repeating the same work and helps keep an unrelated warning from becoming the assumed cause.

9. Request paths

The main request paths are simple enough to map. Sign in uses api to auth. Order details use api to orders. Tracking uses api, orders, and tracking. Documents use api, orders, and documents. Notifications and reports have their own service paths.

The path is the first filter for evidence. If a user says the order page is broken, identify which backend request failed before selecting a service.

Operational notes

- A page-level symptom may hide one failed subrequest.
- What still works narrows the dependency chain.
- Use request IDs when available to correlate API and downstream logs.

Example

When investigating request paths, compare the reported user action with the exact service or dependency that owns it. Record the evidence that supports the conclusion and avoid using unrelated warnings simply because they occur nearby in the logs.

What to record

Record the incident time, affected route, request identifier when available, service state, relevant error code, and the next check. This gives another operator enough context to continue without replaying the whole investigation.

Operational detail

ParcelPilot intentionally separates user journeys across services. This means a broad page-level symptom should be decomposed into the requests that make up the page before a service is blamed.

Dependencies should be followed in the direction the request actually travels. Start at the route, move to the owning service, then inspect the database, snapshot, file store, queue, or provider used by that service.

Example investigation

- Identify the failing request.
- Find the owning service.
- Check its current health and recent logs.
- Inspect only the configuration or data source that belongs to that path.
- Verify with a second request after any change.

This keeps troubleshooting understandable and reduces the chance that unrelated warnings turn into unnecessary changes.

10. Configuration boundaries

Services use environment-style settings for URLs, local database paths, carrier snapshot paths, queue thresholds, and reporting timeouts.

Missing, invalid, and unreachable settings should be treated differently. Tools may report presence and basic validity, but they should not reveal secret values.

Operational notes

- ORDERS_DB_PATH belongs to the orders service.
- CARRIER_SNAPSHOT_PATH belongs to tracking.
- REPORTING_DB_PATH belongs to reporting.

Example

When investigating configuration boundaries, compare the reported user action with the exact service or dependency that owns it. Record the evidence that supports the conclusion and avoid using unrelated warnings simply because they occur nearby in the logs.

What to record

Record the incident time, affected route, request identifier when available, service state, relevant error code, and the next check. This gives another operator enough context to continue without replaying the whole investigation.

Operational detail

ParcelPilot intentionally separates user journeys across services. This means a broad page-level symptom should be decomposed into the requests that make up the page before a service is blamed.

Dependencies should be followed in the direction the request actually travels. Start at the route, move to the owning service, then inspect the database, snapshot, file store, queue, or provider used by that service.

Example investigation

- Identify the failing request.
- Find the owning service.
- Check its current health and recent logs.
- Inspect only the configuration or data source that belongs to that path.
- Verify with a second request after any change.

This keeps troubleshooting understandable and reduces the chance that unrelated warnings turn into unnecessary changes.

11. Observability sources

The training environment includes service status, application logs, configuration snapshots, carrier snapshots, and incident history. Each source answers a different question.

Status is point-in-time. Logs show events over time. Configuration describes expected dependencies. Incident history provides examples but does not prove the present cause.

Operational notes

- Use current evidence before historical similarity.
- Separate known facts from remaining uncertainty.
- A diagnosis should be traceable to evidence.

Example

When investigating observability sources, compare the reported user action with the exact service or dependency that owns it. Record the evidence that supports the conclusion and avoid using unrelated warnings simply because they occur nearby in the logs.

What to record

Record the incident time, affected route, request identifier when available, service state, relevant error code, and the next check. This gives another operator enough context to continue without replaying the whole investigation.

Operational detail

ParcelPilot intentionally separates user journeys across services. This means a broad page-level symptom should be decomposed into the requests that make up the page before a service is blamed.

Dependencies should be followed in the direction the request actually travels. Start at the route, move to the owning service, then inspect the database, snapshot, file store, queue, or provider used by that service.

Example investigation

- Identify the failing request.
- Find the owning service.
- Check its current health and recent logs.
- Inspect only the configuration or data source that belongs to that path.
- Verify with a second request after any change.

This keeps troubleshooting understandable and reduces the chance that unrelated warnings turn into unnecessary changes.